

One VPS, multiple projects

Ubuntu 24.04 deployment guide - no domain required

Deploy the Inventory app using **Nginx, PHP 8.3-FPM, MySQL and Node.js 24**. Give each project its own public port now. Later, map a domain or subdomain to each project without moving its code or database.

Replace **VPS_IP** with your VPS's public IPv4 address, **YOUR_REPOSITORY_URL** with your repository URL, and all password placeholders. These instructions assume a sudo-capable Ubuntu account and projects you own and trust.

Component	Inventory	Second project example
Public HTTP address	http://VPS_IP:8081	http://VPS_IP:8082
Private Node listener	127.0.0.1:3001	127.0.0.1:3002
Project folder	/var/www/inventory	/var/www/project2
MySQL database/user	inventory / inventory	project2 / project2
Web service	inventory-web	project2-web
Queue service	inventory-queue	project2-queue
Future domain	inventory.example.com	project2.example.com

How requests reach this project

```
Browser -> VPS_IP:8081 -> Nginx
/         -> Next.js on 127.0.0.1:3001
/api/*    -> Laravel through PHP-FPM
/storage/* -> Laravel public uploads
/up       -> Laravel health check
```

The website and its API share one origin, including the port. Nginx is shared across projects; each project gets its own server block and application services. Do not expose Node ports 3001/3002 or MySQL port 3306 publicly.

The repository contains **web/** (Next.js), **inventory-api/** (Laravel), **mobile/** (Expo client) and **salepro-clone/** (reference code, not deployed).

Read this before starting

Use HTTP only for initial connectivity checks. It does not encrypt passwords or tokens. Before using real accounts over the Internet, enable the optional public-IP HTTPS setup on pages 9-10, or connect your domain using page 11.

Install shared packages once. Repeat only the project-specific steps for each additional app. Check existing ports with **sudo ss -ltnp** before choosing new ones. Keep existing Nginx sites and services in place.

This guide provides operational separation for your own projects. A shared deploy account and PHP-FPM user do not isolate mutually untrusted applications.

Prepare the VPS

1. Install shared dependencies once

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y \
  nginx mysql-server git curl unzip acl cron \
  php8.3-cli php8.3-fpm php8.3-mysql php8.3-sqlite3 \
  php8.3-mbstring php8.3-xml php8.3-curl php8.3-zip \
  php8.3-bcmath php8.3-intl php8.3-gd composer
curl -fsSL https://deb.nodesource.com/setup_24.x \
  -o /tmp/nodesource_setup.sh
sudo bash /tmp/nodesource_setup.sh
sudo apt install -y nodejs
sudo systemctl enable --now nginx mysql php8.3-fpm cron
php -v
node -v
composer --version
```

On an existing VPS, schedule package upgrades and service restarts around the other projects. The remaining application setup uses PHP 8.3 and Node.js 24.

2. Create a deployment account and fetch the project

Skip account creation if you already have a suitable deploy account. Run the remaining commands as **deploy**, using `sudo` where shown.

```
sudo adduser deploy
sudo usermod -aG sudo deploy
sudo mkdir -p /var/www/inventory
sudo chown deploy:deploy /var/www/inventory
sudo -iu deploy
git clone YOUR_REPOSITORY_URL /var/www/inventory
```

Use your Git credentials or an SSH deploy key if the repository is private.

3. Create this project's database

Run **sudo mysql**, then enter this SQL with a new password:

```
CREATE DATABASE inventory
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'inventory'@'localhost'
  IDENTIFIED BY 'REPLACE_WITH_A_LONG_RANDOM_PASSWORD';
GRANT ALL PRIVILEGES ON inventory.*
  TO 'inventory'@'localhost';
EXIT;
```

Create a different database and database user for each project. MySQL can be shared; application credentials should not be shared.

Configure Laravel

4. Set the environment for IP-and-port access

```
cd /var/www/inventory/inventory-api
cp .env.example .env
nano .env
```

Set the following values, preserving the other settings. Replace VPS_IP with the actual address; do not leave it as literal text.

```
APP_NAME="Inventory"
APP_ENV=production
APP_DEBUG=false
APP_URL=http://VPS_IP:8081
LOG_LEVEL=warning
DB_CONNECTION=mysql
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=inventory
DB_USERNAME=inventory
DB_PASSWORD="REPLACE_WITH_A_LONG_RANDOM_PASSWORD"
SESSION_DRIVER=database
SESSION_DOMAIN=null
SESSION_COOKIE=inventory_session
SESSION_SECURE_COOKIE=false
CACHE_STORE=database
CACHE_PREFIX=inventory_cache_
QUEUE_CONNECTION=database
MAIL_MAILER=log
```

Cookies are shared by hostname across ports. Use a different **SESSION_COOKIE** for each project. Keep SESSION_DOMAIN unset/null; do not put a port or URL in it. Set SESSION_SECURE_COOKIE=true when enabling HTTPS.

The example environment contains Mailtrap settings. Replace them with your own SMTP credentials when enabling email. **MAIL_MAILER=log records messages locally; it does not deliver them.**

Install dependencies and set permissions

```
composer install --no-dev --prefer-dist --optimize-autoloader
composer check-platform-reqs --no-dev
php artisan key:generate
sudo chgrp www-data .env
chmod 640 .env
sudo setfacl -R -m u:deploy:rwX,u:www-data:rwX storage bootstrap/cache
sudo setfacl -dR -m u:deploy:rwX,u:www-data:rwX storage bootstrap/cache
```

Existing installation: preserve its .env, APP_KEY, database and uploaded files. Generate an APP_KEY only for a new installation. Use a different APP_KEY for each independently initialized project.

Initialize the app and build the website

5. Initialize a new database and replace the demo login

```
cd /var/www/inventory/inventory-api
php artisan migrate --force
php artisan db:seed --force
php artisan storage:link
php artisan tinker
```

The seeder creates roles, settings, sample business records and **admin@example.com / password**. Replace this login before opening access. Inside Tinker:

```
$user = App\Models\User::where('email', 'admin@example.com')->firstOrFail();
$user->forceFill([
    'email' => 'you@example.com',
    'password' => Illuminate\Support\Facades\Hash::make(
        \Laravel\Prompts\password('New admin password')
    ),
])->save();
exit
```

Back in the shell, run **php artisan optimize**. If migrating existing data, restore the database and uploads instead of seeding. Do not rerun the main seeder during routine updates; it overwrites some settings.

6. Build Next.js

```
cd /var/www/inventory/web
cat > .env.production.local <<'ENV'
NEXT_PUBLIC_API_URL=http://VPS_IP:8081/api
ENV
npm ci
npm run build
```

Replace VPS_IP in the file before building. NEXT_PUBLIC_* values are embedded at build time. Rebuild after changing the IP, port, scheme or domain.

Create **/etc/systemd/system/inventory-web.service** using sudo nano:

```
[Unit]
Description=Inventory Next.js website
After=network.target

[Service]
Type=simple
User=deploy
Group=deploy
WorkingDirectory=/var/www/inventory/web
Environment=NODE_ENV=production
ExecStart=/usr/bin/node /var/www/inventory/web/node_modules/next/dist/bin/next \
    start --hostname 127.0.0.1 --port 3001
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Run the queue and scheduler

7. Create the queue worker

This API queues invoice emails and runs **sales-invoice-emails:recover** every minute. It needs both a worker and the Laravel scheduler.

Create **/etc/systemd/system/inventory-queue.service** using sudo nano:

```
[Unit]
Description=Inventory Laravel queue worker
After=network.target mysql.service

[Service]
Type=simple
User=www-data
Group=www-data
WorkingDirectory=/var/www/inventory/inventory-api
ExecStart=/usr/bin/php artisan queue:work --sleep=3 --tries=3 --timeout=60
Restart=always
RestartSec=5
KillSignal=SIGTERM
TimeoutStopSec=120

[Install]
WantedBy=multi-user.target
```

Start both application services:

```
sudo systemctl daemon-reload
sudo systemctl enable --now inventory-web inventory-queue
```

8. Add the scheduled task

Create **/etc/cron.d/inventory** using sudo nano. Paste these two lines, with a newline at the end of the file:

```
INVENTORY_API=/var/www/inventory/inventory-api
* * * * * www-data cd $INVENTORY_API && /usr/bin/php artisan schedule:run >> storage/logs/scheduler.log 2>&1

sudo chmod 644 /etc/cron.d/inventory
cd /var/www/inventory/inventory-api
php artisan schedule:list
sudo systemctl status inventory-web inventory-queue --no-pager
```

Give additional projects their own cron files, application paths and queue services. A scheduler or worker pointed at Inventory will not process another project's jobs.

Workers are long-running processes. Restart the relevant project's worker when deploying new application code or changing its cached configuration.

Route Inventory through Nginx

9. Create a dedicated server block on port 8081

Run **sudo nano /etc/nginx/sites-available/inventory** and paste:

```
server {
    listen 8081;
    server_name _;
    root /var/www/inventory/inventory-api/public;
    client_max_body_size 24M;

    location = /api { rewrite ^ /index.php last; }
    location ^~ /api/ { rewrite ^ /index.php last; }
    location = /up { rewrite ^ /index.php last; }
    location /storage/ { try_files $uri =404; }

    location = /index.php {
        internal;
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root/index.php;
        fastcgi_param HTTP_AUTHORIZATION $http_authorization;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
    }
    location ~ \.php(?:/|$) { return 404; }
    location ~ /\.(!well-known).* { deny all; }

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $http_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_buffering off;
    }
}
```

Host \$http_host preserves the public port. This matters for IP-and-port URLs. Keep the Laravel document root at inventory-api/public, not the repository root. Only the front controller executes PHP.

Enable this site without removing existing sites:

```
sudo ln -s /etc/nginx/sites-available/inventory \
    /etc/nginx/sites-enabled/inventory
sudo nginx -t
sudo systemctl reload nginx
```

Align PHP upload limits; this PHP-FPM setting affects other apps using the same PHP version:

```
sudo tee /etc/php/8.3/fpm/conf.d/99-inventory.ini >/dev/null <<'INI'
upload_max_filesize = 20M
post_max_size = 24M
INI
sudo systemctl reload php8.3-fpm
```

Firewall, verification and mobile access

10. Open only the public application port

For SSH on the standard port:

```
sudo ufw allow OpenSSH
sudo ufw allow 8081/tcp
sudo ufw enable
sudo ufw status
```

If SSH uses a custom port, allow it before enabling UFW. Preserve rules needed by existing projects. Allow TCP 8081 in the VPS provider's firewall too. Do not open 3001 or 3306.

11. Verify initial HTTP routing

```
sudo systemctl status inventory-web inventory-queue --no-pager
curl -I http://VPS_IP:8081
curl -i http://VPS_IP:8081/up
curl -i -H 'Accept: application/json' http://VPS_IP:8081/api/me
```

Expected: the website returns success or a redirect; /up returns HTTP 200; /api/me returns HTTP 401 without a login token. These checks confirm routing, not the full business workflow.

Enable HTTPS before sending real credentials over the Internet. Then test login, product creation, uploads and invoice generation. Configure real SMTP and notification settings before testing email delivery.

Useful logs:

```
sudo journalctl -u inventory-web -n 100 --no-pager
sudo journalctl -u inventory-queue -n 100 --no-pager
tail -n 100 /var/www/inventory/inventory-api/storage/logs/laravel.log
sudo tail -n 100 /var/log/nginx/error.log
```

If the browser tries localhost:8000, check web/.env.production.local and rebuild. If another app appears, check the public port and Nginx server block. A 502 usually means Next.js or PHP-FPM is unavailable. Confirm that generated image URLs include :8081.

12. Connect the Expo mobile app

For initial configuration, the API address is:

```
EXPO_PUBLIC_API_URL=http://VPS_IP:8081/api
```

After enabling IP HTTPS, change it to **https://VPS_IP:8081/api** and rebuild. Native release builds may restrict cleartext HTTP; use the trusted HTTPS endpoint for production. Later, rebuild with your domain's API URL.

Use the public API address, never 127.0.0.1 or the private Node port, in the phone app.

Add projects and deploy updates

13. Add project 2 without disturbing Inventory

For another copy of this stack, repeat the application steps with these replacements. For another technology stack, keep the port-and-service pattern but use that app's own runtime and database setup.

Setting	Inventory	Project 2
Folder	/var/www/inventory	/var/www/project2
Public Nginx port	8081	8082
Private Next.js port	3001	3002
Nginx config filename	inventory	project2
systemd services	inventory-web / inventory-queue	project2-web / project2-queue
Cron filename	/etc/cron.d/inventory	/etc/cron.d/project2
DB name and DB user	inventory	project2
SESSION_COOKIE	inventory_session	project2_session
CACHE_PREFIX	inventory_cache_	project2_cache_

Create a separate .env and APP_KEY. Update every WorkingDirectory, ExecStart, Nginx root and scheduler path. For project 2, use APP_URL=http://VPS_IP:8082 and NEXT_PUBLIC_API_URL=http://VPS_IP:8082/api before its build. Open **8082/tcp** only when it is ready.

Nginx can serve both projects using the same server_name _ because they listen on different ports. If both listen on the same IP and port with the same server name, Nginx cannot distinguish them by project folder.

Share installed runtimes and MySQL, but keep each app's dependencies, databases, secrets, uploads and service names separate. Plan RAM for all services plus builds; build projects one at a time on a small VPS.

14. Update only the affected project

1. Back up that project's MySQL database, inventory-api/storage/app and .env. Keep backups off the VPS and periodically verify restoration.
2. In its API directory, run php artisan down. Stop only its web and queue services for a maintenance window.
3. Pull the updated code in its repository directory.
4. In inventory-api/, run composer install --no-dev --prefer-dist --optimize-autoloader, then php artisan migrate --force and php artisan optimize.
5. In web/, run npm ci and npm run build.
6. Start its web and queue services; run php artisan up in its API directory. Test its public URL.

Do not run migrate:fresh or the main seeder during routine updates. Keep production .env files, APP_KEY and uploads. If a build or migration fails, resolve it or restore the release before reopening access.

Avoid restarting all Node or PHP processes just to update one app. A validated **Nginx reload** applies routing changes without stopping unrelated applications.

Optional now: HTTPS for your public IP

15. Obtain an IP certificate without a domain

Let's Encrypt supports public-IP certificates. The documented Certbot webroot flow requires **Certbot 5.4+** and uses six-day certificates. Automatic renewal and an Nginx reload hook are essential. A stable public IP and inbound TCP 80 are required for this HTTP-01 validation flow. [1]

Install Certbot once for the VPS:

```
sudo apt install -y snapd
sudo snap install --classic certbot
sudo /snap/bin/certbot --version
sudo mkdir -p /var/www/acme/.well-known/acme-challenge
sudo ufw allow 80/tcp
```

Also allow port 80 in the provider firewall. Create a shared challenge server using **sudo nano /etc/nginx/sites-available/ip-acme**:

```
server {
    listen 80;
    server_name VPS_IP;
    location ^~ /.well-known/acme-challenge/ {
        root /var/www/acme;
        default_type text/plain;
        try_files $uri =404;
    }
    location / { return 404; }
}
```

Replace VPS_IP with the actual IP. If an existing Nginx block already handles that IP on port 80, add this challenge location to that block instead of creating a duplicate. If another web server owns port 80, integrate the challenge there first; do not stop unrelated projects.

```
sudo ln -s /etc/nginx/sites-available/ip-acme \
/etc/nginx/sites-enabled/ip-acme
sudo nginx -t
sudo systemctl reload nginx
```

Skip the symlink if you edited an existing block. Request the certificate, replacing VPS_IP:

```
sudo /snap/bin/certbot certonly --webroot \
--webroot-path /var/www/acme \
--preferred-profile shortlived \
--ip-address VPS_IP \
--cert-name vps-ip \
--deploy-hook 'systemctl reload nginx'
```

Follow Certbot's account prompts. A certificate validates the IP, not a port; the same IP certificate can be used by all these project ports. This flow uses certonly and manual Nginx configuration; it does not depend on Certbot's Nginx installer supporting IP certificates.

Enable IP HTTPS and renewal

16. Switch Inventory's public port to TLS

In `/etc/nginx/sites-available/inventory`, replace **listen 8081**; with:

```
listen 8081 ssl;
ssl_certificate /etc/letsencrypt/live/vps-ip/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/vps-ip/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
```

Keep its routes and private upstream unchanged. This converts port 8081 to HTTPS; use **https://VPS_IP:8081** from now on. Plain HTTP on this port is no longer the supported URL. Repeat for each project port when ready.

Update `inventory-api/.env`:

```
APP_URL=https://VPS_IP:8081
SESSION_SECURE_COOKIE=true
```

Update `web/.env.production.local`:

```
NEXT_PUBLIC_API_URL=https://VPS_IP:8081/api
```

Apply these changes in a maintenance window:

```
sudo systemctl stop inventory-web
cd /var/www/inventory/web
npm run build
cd /var/www/inventory/inventory-api
php artisan optimize
sudo nginx -t
sudo systemctl reload nginx
sudo systemctl restart inventory-web inventory-queue
curl -i https://VPS_IP:8081/up
```

Laravel's `fastcgi_params` include the HTTPS state when Nginx terminates TLS. Verify that uploaded-image URLs use https and retain the public port.

17. Verify automatic renewal

```
sudo /snap/bin/certbot renew --dry-run
sudo systemctl list-timers --all | grep -i certbot
```

Confirm the renewal timer exists and check that the renewal configuration retains the deploy hook to reload Nginx. Keep TCP 80 and the challenge location available for renewals. Monitor failures; these IP certificates last only six days. If the VPS IP changes, obtain a certificate for the new IP and update application URLs. [1]

References for IP access

[1] [Let's Encrypt: IP certificates and Certbot webroot setup](#)

[Nginx: request routing by address, port and server name](#)

[Next.js: self-hosting and build-time public environment variables](#)

Later: connect your domains

18. Keep the app; change its public address

When you own a domain, point an **A record** such as `inventory.example.com` to the VPS IPv4 address. Point `project2.example.com` to the same IP. Add AAAA records only if working IPv6 routing is configured. Each project can also use a completely separate domain.

Keep `/var/www/inventory`, its database, uploads, systemd services and private port 3001. Keep the old IP-and-port server block temporarily while switching clients.

Create `/etc/nginx/sites-available/inventory-domain` by copying the Inventory server block. In this new copy, replace its `listen` and `server_name` lines with the following and remove any IP certificate directives from the copy:

```
listen 80;
server_name inventory.example.com;
```

Keep the existing Laravel root, routes, FastCGI configuration and Next.js proxy to `127.0.0.1:3001`. Enable the new file, validate Nginx and obtain a domain certificate:

```
sudo ln -s /etc/nginx/sites-available/inventory-domain \
    /etc/nginx/sites-enabled/inventory-domain
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo nginx -t
sudo systemctl reload nginx
sudo /snap/bin/certbot --nginx -d inventory.example.com --redirect
```

Allow 80/443 in the provider firewall. If you skipped IP HTTPS, install the Certbot snap first as shown on page 9. With a domain certificate, Certbot's Nginx installer can configure HTTPS and the HTTP redirect.

19. Update URLs and rebuild clients

```
# inventory-api/.env
APP_URL=https://inventory.example.com
SESSION_SECURE_COOKIE=true
SESSION_DOMAIN=null
# web/.env.production.local
NEXT_PUBLIC_API_URL=https://inventory.example.com/api
# mobile build environment
EXO_PUBLIC_API_URL=https://inventory.example.com/api
```

Run **php artisan optimize**, rebuild the web app with **npm run build**, and restart `inventory-web` and `inventory-queue` in a maintenance window. Rebuild/distribute the mobile app with its new URL. Test login, uploads, invoice links and `/up` through the domain. Users should expect to sign in again at the new origin.

After all clients move, disable the old Inventory port server block and remove its 8081 firewall allowance. Keep other projects' ports until they migrate. Do not remove the shared IP certificate or port-80 challenge route while any project still uses them. Domains share ports 80/443; their distinct `server_name` values choose the project.

[Laravel deployment](#) | [Laravel queues](#) | [Certbot Nginx setup](#)

Prepared 16 September 2026 from the repository and previous guide. Commands have not been executed on your VPS. The editable Markdown source is saved beside this PDF so deleting temporary files will not remove the guide's source.